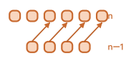
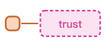
概念	维度	编码	
自指数据 哪种数据引用了自身？一个节点里放什么？	数据	**列表** HtDP DCIC rest 仍然是一个列表。	
		树 MIT 6.102 ×2 孩子仍然是树。	
		套娃 HtDP 图画里的物体装着更小的自己。	
	节点内容	**具体数据** DCIC MIT 6.102 节点里是输入本身的值。	[1, 4, 3]
		带参数的调用 SICP LeetCode ×3 按代码写出的调用，参数是具体值。	fib 3
		状态元组 LeetCode ×3 只写出变化的参数。	(i, k)
		部分求值表达式 SICP HtDP LeetCode 算到一半的表达式，像代换轨迹里那样。	(* 6 (f 5))
基本情况 递归停下的地方有没有画出来？怎么画？	标记方式	**有标记** LeetCode ×4 作为基本情况标出或高亮。	
		画到但没标记 LYAH 画出了最后一次调用，但没有特别标出。	f(1) → f(0)
		死路叶子 LeetCode 失败的分支让递归停下。	

概念	维度	编码	
分解 更小的子问题怎么排布？画出多少次调用？	结构	**嵌套包含** 被调用者画在调用者内部。	盒中盒 HtDP 
			括号嵌套 LYAH LeetCode ×2 f(a(b(c)))
		树（节点–连线） 调用和它的子调用用连线相连。	调用树 SICP LeetCode ×3 
			状态树 LeetCode ×2 
			选择树 LeetCode 
			结果树 LeetCode ×6 
		线性链 由节点和指针组成的链表。	盒子与指针 HtDP LeetCode 
			指针重连 LeetCode ×8 
		栈 LeetCode ×2 按深度堆起来的栈帧。	
		表格 HtDP LeetCode 参数、中间值和结果列成表。	
		代换轨迹 SICP HtDP ×3 LeetCode 表达式不断改写，直到得出值。	
		缩进阶梯 DCIC LeetCode 用缩进表示深度。	
		输入划分 LeetCode 把输入切成几块并框起来。	
		规模映射 LeetCode ×3 规模 n 与规模 n−1 对齐。	
		隐喻 / 图画 LYAH HtDP LeetCode 套娃、镜像、螺旋等。	
	选择	**选择标签** LeetCode ×2 在连线上写出所做的选择。	
		剪枝 / 失败 + 图例 LeetCode ×3 死路分支，用图例说明。	
	省略（调用）	**完全展开** SICP LeetCode ×2 输入较小，每次调用都画出来。	
		有规律省略 CLRS 画几层，规律可以推出来。	
		无规律省略 LeetCode 用“…”省略，看不出规律。	
		延迟调用 子调用先不展开。	轨迹中的未展开项 HtDP f(rest…)
			“已完成”框 LeetCode ×2 done
		信任子调用 LeetCode ×5 直接使用子调用的结果，不画它的内部（leap of faith）。	

概念	维度	编码	
传回合并 结果怎么传回来？画出多少次返回？	流向	**只向上** HtDP 只画结果向根合并。	
		往返 SICP LeetCode ×7 下到基本情况，再返回上来。	
		原地改写 SICP HtDP ×2 时间顺序就是从上到下的行。	f(3) ⇒ 3·f(2) ⇒ 3·2·f(1) ⇒ 6
	标注	**返回值** LeetCode ×3 在回传路上写出值。	
		合并步骤 SICP HtDP LeetCode ×2 画出子结果怎样合并。	
		遍历路径 SICP 画出求值的顺序。	
		调用步骤编号 LeetCode ×2 给每一帧或每一步编号。	
	省略（返回）	**画出全部返回** SICP ×2 HtDP ×3 LeetCode ×7 每次调用的结果都画出传回。	
		画出部分返回 LeetCode 只画一部分结果，比如最后的结果。	
		不画返回 LeetCode ×4 只画调用；对递归过程来说缺了回程。	

LYAH Learn You a Haskell（Haskell 教材）　SICP 计算机程序的构造和解释（SICP）　HtDP 程序设计方法（HtDP）　DCIC A Data-Centric Introduction to Computing　CLRS 算法导论（CLRS）

MIT 6.102 MIT 6.102 课程讲义　LeetCode LeetCode 题解，vote ⩾ 5　×n 例子数

图 X：递归图示的设计空间（三张表）。每个递归概念下列出它的画法维度。